CLIENT-SIDE DLP ARCHITECTURE

Shadow AI Prompt Guardian SENTINEL v5.0

Complete Project Breakdown, Technical Architecture, Evaluation Benchmarks & Comprehensive Viva Examination Guide

---

TECHNOLOGY STANDARD

Chrome Manifest V3 · Service Worker

DETECTION PARADIGM

150+ Signatures · Shannon Entropy · Luhn Check

PLATFORM COVERAGE

25+ AI Platforms (ChatGPT, Claude, Gemini, etc.)

EVALUATION METRIC

100% Accuracy · 1.00 F1 Score · 0% FPR (95 Prompts)

Project Scope: Mini-Project / Final Year Project Guide

Zero-Trust Client-Side Architecture

📑 Table of Contents

Section	Topic Title	Focus Area
1	Project Overview	Title, problem statement, real-world relevance, target users
2	Objectives	Major technical goals of client-side prompt DLP
3	Features & Capabilities	Detailed matrix of every built-in capability
4	Technology Stack	Frontend, runtime, storage, libraries, rationale
5	Complete Project Structure	Folder hierarchy, file purposes, dependency mappings
6	System Workflow	Step-by-step lifecycle from keystroke to blocking/redaction
7	System Architecture	5-layer zero-trust architecture diagram & data flow
8	Database & Storage	chrome.storage.local schema, keys, and masking
9	Code Explanation	Deep-dive into detection engine, interceptors, React sync
10	Algorithms & Logic	Shannon entropy math, Luhn checksum, hash chaining
11	API & Extension IPC	Message passing channels, request/response protocols
12	Authentication & Security	Pre-TLS defense, zero data exfiltration, storage isolation
13	User Interface Breakdown	HUD banner, explainability drawer, Mission Control popup
14	Project Setup & Execution	Installation, Node test commands, Chrome load instructions
15	Testing & Quality Assurance	121 unit tests, 95 evaluation prompts, confusion matrix
16	Results & Current Status	Implemented features, verified benchmarks, partial modules
17	Limitations	Current boundaries and client-side constraints
18	Future Enhancements	Wasm OCR, enterprise SIEM webhooks, cross-browser ports
19	College Project Report Content	Complete ready-to-use mini project report text
20	Viva Preparation (30 Q&A)	30 examiner questions with concise, confident answers
21	2-Minute Faculty Pitch	Natural spoken script for quick faculty demonstration
22	5-Minute Presentation Script	Complete slide-by-slide project presentation speech
23	Codebase Verification Audit	Truth matrix, implementation checks, cross-file notes
★	WHAT I MUST KNOW FOR VIVA	Top 20 high-yield concepts to speak fluently

1. Project Overview

Project Title: Shadow AI Prompt Guardian — SENTINEL (v5.0.0 / v4.0 "NEXUS")

Project Purpose: A client-side Data Loss Prevention (DLP) browser extension that inspects, sanitizes, and blocks sensitive credentials, API keys, Personally Identifiable Information (PII), and confidential trade secrets in real time before they leave the browser and reach third-party AI models (ChatGPT, Claude, Gemini, Copilot, Perplexity, etc.).

Problem Statement: Employees and developers frequently paste proprietary source code, customer records, database credentials, and production API keys into public AI chatbots (called *"Shadow AI"* usage). Once submitted over TLS, this sensitive information is transmitted to external servers, risking data leaks, regulatory fines (GDPR, HIPAA, PCI-DSS), and credential compromise.

Why This Project Was Created: Existing corporate DLP solutions require complex network proxies, VPN agents, or server-side relays that violate end-to-end privacy and introduce latency. SENTINEL was created to run **100% locally in the browser client**—zero telemetry, zero cloud dependencies, and zero external network calls.

Main Idea in Simple Words: Think of SENTINEL as a **smart security guard standing between your keyboard and AI chatbots**. Before your prompt gets sent, SENTINEL scans the text inside your browser. If you accidentally included an AWS key, password, customer email list, or confidential codename, SENTINEL intercepts the submission, shows a warning banner with an explainability panel, and lets you auto-redact the secrets with safe placeholders in one click.

Target Users:

- Software engineers and DevOps professionals working with cloud credentials and API keys.
- Enterprise employees handling proprietary business documents, customer PII, or patient health data.
- Security & Compliance Officers looking for lightweight, zero-trust client-side DLP enforcement.

Real-World Use Case: A developer is debugging an authentication script and pastes a code snippet into ChatGPT containing `AKIAIOSFODNN7EXAMPLE` and a database connection string. SENTINEL intercepts the `fetch` / `submit` event, halts the network request, displays a critical warning HUD, and replaces the secrets with `[API_KEY]` and `[PASSWORD]` when the developer clicks **"Auto-Redact & Resubmit"**.

2. Objectives

1. **Pre-Transmission Interception:** Catch user prompts in the browser DOM and network layer before TLS encryption and transmission occur.
2. **Multi-Layer Detection Engine:** Combine deterministic regex patterns (150+ signatures), mathematical Shannon entropy calculation, Luhn algorithm card validation, and custom keyword dictionaries.
3. **Zero Data Exfiltration & Privacy-First Architecture:** Ensure 100% of detection and audit storage happens locally inside the browser (`chrome.storage.local`), never transmitting prompt data to any external server.
4. **Seamless Workflow Integration (Auto-Redaction):** Allow developers to sanitize prompts in-place with safe placeholders (`[API_KEY]`, `[REDACTED]`) and trigger synthetic React events so the chatbot UI stays fully functional without manual re-typing.
5. **Contextual Risk Scoring & Explainability:** Distinguish educational queries (e.g., *"What is an AWS key?"*) from actual leaks, assign a 0–100 risk score, and provide plain-English explanations without exposing raw secrets.
6. **Tamper-Proof Audit Logging:** Record local security incident entries linked via SHA-256 cryptographic hash chains for enterprise auditability.

3. Features & Capabilities

Feature	What It Does	Implementation File	Logic Used
150+ Secret Signatures	Detects cloud, AI, payment, and database credentials	`js/detection-engine.js`	`API_KEY_PATTERNS` regex dictionary (AWS, GitHub, OpenAI, Stripe, GCP, etc.).
Shannon Entropy Secret Detection	Catches unknown/custom high-entropy secret tokens	`js/detection-engine.js`	Computes $H(X) \ge 4.0$ on tokens ≥ 20 characters.
PII & Luhn Card Validation	Detects emails, phones, SSNs, IBANs, and credit cards	`js/detection-engine.js`	Matches PII patterns and filters card PANs with MOD-10 `luhnCheck()`.
5-Vector Interception Engine	Intercepts submissions across all browser pathways	`js/content.js` & `js/main-world-intercept.js`	Hooks `fetch`, `XHR`, Form submit (capture phase), Enter keydown, Send button.
Holographic HUD Warning Banner	Displays threat alert with dynamic severity badges	`js/content.js` & `css/content.css`	Glassmorphism HUD, animated shield icon, risk meter, action buttons.
One-Click Auto-Redaction	Replaces sensitive tokens with placeholders	`js/detection-engine.js` & `js/content.js`	Splices tokens in descending index order, sets native descriptor, dispatches React events.
Bulk Data Leak Detection	Identifies database/spreadsheet dump exports	`js/detection-engine.js`	Flags ≥ 5 same type or ≥ 10 mixed PII as Critical data dumps.
Explainable Detection Panel	Explains *why* content was flagged without leaking secrets	`js/content.js`	Click-to-open drawer displaying rule name, reason, and character position.
Real-Time Mirror Overlay	Highlights risky text as the user types (300ms debounce)	`js/content.js`	Mirrors textarea layout with transparent overlay and colored `<mark>` spans.
Live Floating Risk Indicator	Shows a live risk pill at the bottom corner (400ms debounce)	`js/content.js`	Debounced risk scoring: Safe $\to$ Low $\to$ High $\to$ Danger.
Obfuscation Normalizer	Defeats leak bypass attempts (leet-speak, zero-width chars)	`js/detection-engine.js`	Strips invisible unicode, maps lookalikes, collapses spaced characters.
Base64 Decode-and-Scan	Catches hidden encoded credentials	`js/detection-engine.js`	Regex base64 matching (≥ 32 chars), `atob()` decoding, and re-scanning.
Compliance Presets	Specialized modes for HIPAA, PCI-DSS, GDPR, SOC 2	`js/detection-engine.js`	Applies domain pattern sets (NPI, MRN, ICD-10) and 1.2x–1.5x risk multipliers.

Context Window Slow-Drip Scan	Tracks cumulative risk across consecutive chat turns	`js/detection-engine.js`	Stores last 5 messages in conversation memory to catch split secrets.
API Key Quick-Rotation Links	Provides direct links to revoke/rotate exposed keys	`js/detection-engine.js`	Maps flagged key types to AWS IAM, GitHub Tokens, OpenAI API keys, etc.
Conversation Chat Scanner	Scans historical messages on the current AI chat page	`js/content.js`	DOM query selectors extracting past bubbles and checking for past leaks.
Tamper-Proof Audit Log	Maintains an immutable audit trail of blocked attempts	`js/background.js`	Computes SHA-256 hash chains linking each block entry to previous entries.
Mission Control Popup	Toolbar popup overview	`popup.html` & `js/popup.js`	240° SVG Exposure Gauge, risk history sparkline, recent blocks.
Guardian Center Dashboard	Comprehensive settings and rule management page	`options.html` & `js/options.js`	Sensitivity slider, preset toggles, custom regex/terms, JSON export.
Public Developer API	Enables external programmatic scanning	`js/detection-engine.js`	Exposes `window.SentinelAPI` (`scan` , `redact` , `entropy`).

4. Technology Stack

- **Frontend:** Vanilla HTML5, Vanilla JavaScript (ES6+), Vanilla CSS3 (Custom design system with tokens, CSS Grid, Flexbox, glassmorphism, keyframe animations).
  Rationale: Extensions must be ultra-lightweight. Zero-framework design guarantees instant (<1ms) load time and no build pipeline.
- **Backend / Runtime:** Chrome Extensions **Manifest V3** with Background Service Worker (`js/background.js`).
  Rationale: Manifest V3 is the modern, secure standard for Chromium browsers replacing persistent background pages with event-driven service workers.
- **Database / Storage:** `chrome.storage.local` (Client-side browser storage API).
  Rationale: Stores configuration, custom rules, and masked audit logs locally on the user's hard drive without network overhead.
- **Programming Languages:** JavaScript (Node.js for evaluation harness/tests, Browser JS for runtime).
- **Security & Cryptography:** SHA-256 hashing via `crypto.subtle.digest` for hash chaining; Luhn algorithm ($O(n)$ check digit verification).
- **AI/ML Technologies (Phase 2 Experimental):** Lightweight client-side Logistic Regression Classifier (`js/phases/ml-classifier.js`) with TF-IDF feature extraction and gradient descent training.

5. Complete Project Structure

```
shadow-ai-prompt-guardian/
├── manifest.json                  # Manifest V3 manifest, permissions, host rules, content scripts
├── popup.html                     # Toolbar popup UI (Mission Control)
├── options.html                   # Options / Settings dashboard (Guardian Center)
├── eval-harness.js                # Automated benchmark harness (95 test prompts)
├── results.json                   # Machine-readable evaluation benchmark metrics
├── results.md                     # Human-readable evaluation report
├── DECISIONS.md                   # Architecture & engineering design decisions record
├── PROJECT_CHECKLIST.md           # Requirement tracking & implementation checklist
├── VERIFICATION_REPORT.md         # Comprehensive verification and quality assurance report
├── css/
│   └── content.css                # Cinematic warning HUD, neon highlights, overlays, toast styles
├── js/
│   ├── background.js              # Background service worker (metrics, badge, SHA-256 audit chain)
│   ├── content.js                 # Content script (DOM observer, HUD injector, input event interceptor)
│   ├── detection-engine.js        # Core detection engine (150+ regex, Shannon entropy, Luhn, scoring)
│   ├── main-world-intercept.js    # Injected main-world script (native fetch & XHR monkeypatching)
│   ├── popup.js                   # Mission Control logic (SVG gauge, sparkline, recent blocks)
│   ├── options.js                 # Guardian Center logic (settings, presets, custom rules, audit export)
│   └── phases/
│       └── ml-classifier.js       # Phase 2 experimental in-browser Logistic Regression classifier
├── icons/                         # 16x16, 48x48, 128x128 extension icons
└── tests/
    ├── test-detection-engine.js   # 121 unit tests covering all detection features & edge cases
    ├── test-prompts.json          # 95 synthetic & realistic test prompts with ground truth labels
    └── qa-checklist.js          # Static & regression test suite
```

6. System Workflow & 7. Architecture

System Workflow:

1. **User Interaction:** User types or pastes text into an AI chatbot input.

2. **Passive Monitoring:** 300ms debounced mirror overlay displays colored inline highlights; 400ms debounced floating badge updates live risk score.

3. **Submission Interception:** When Enter is pressed or a network request is dispatched, SENTINEL catches the event.

4. **Multi-Layer Analysis:** Normalized text is evaluated through 150+ regexes, Shannon entropy, Luhn validation, and bulk data detectors.

5. **Decision:** If Critical or Risk Score $\ge$ Threshold, request is aborted, HUD banner is displayed, and audit log is recorded with SHA-256 chaining.

6. **Auto-Redact:** One-click button replaces secrets with safe tokens and synchronizes React's internal state tree.

8. Database & Local Storage

Storage Key	Type	Purpose	Sensitive Data?
`sapg_protectionEnabled`	Boolean	Master protection toggle	No
`sapg_sensitivityThreshold`	Number	Numeric threshold (10–100, default: 30 / 50)	No
`sapg_compliancePresets`	Array<String>	Active compliance modes (e.g. `["hipaa"]`)	No
`sapr_customTerms`	Array<String>	User-defined organization keywords	No
`sapr_customRegex`	Array<String>	User-defined custom regex rules	No
`sapg_blockedAttempts`	Array<Object>	Recent blocks displayed in popup	Masked Only
`sapg_audit_log`	Array<Object>	Tamper-proof chained audit history	Masked Only
`sapg_audit_chain`	String (Hex)	Current 64-char SHA-256 chain head	No

9. Code Explanation & 10. Algorithms

1. Shannon Entropy Formula (H)

Used to detect unknown high-entropy random keys without known prefixes:

$$H(X) = -\sum_{i=1}^{n} P(x_i) \log_2 P(x_i)$$

Tokens with length ≥ 20 and $H \ge 4.0$ are flagged as `high_entropy_token`.

2. Luhn Algorithm Checksum (MOD-10)

Verifies credit card candidate strings to prevent false positives:

- Reverses numeric string and doubles every second digit.
- If doubled value > 9, subtracts 9. Sums all digits.
- If `sum % 10 === 0`, the candidate is a valid payment card number.

3. Cryptographic Hash Chaining

$$H_k = \text{SHA-256}(H_{k-1} \parallel \text{Timestamp} \parallel \text{Categories} \parallel \text{RiskScore})$$

Maintains an immutable chain linking every audit log entry to its predecessor.

4. Auto-Redaction Reverse Splicing

```
// Sort findings in descending order of start position to prevent index shifting
const sorted = [...findings].sort((a, b) => b.start - a.start);
let redacted = text;
for (const f of sorted) {
  const placeholder = PLACEHOLDERS[f.category] || '[REDACTED]';
  redacted = redacted.substring(0, f.start) + placeholder + redacted.substring(f.end);
}
```

11. Extension IPC & 12. Security Model

- **Main-World $\leftrightarrow$ Content Script Bridge:** Communicates via `window.postMessage` (`SAPG_CHECK_PROMPT` and `SAPG_BLOCK_DECISION`).
- **Content Script $\leftrightarrow$ Background Worker:** Communicates via `chrome.runtime.sendMessage` (`SAPG_BLOCK_OCCURRED`, `SAPG_GET_METRICS`).
- **Pre-TLS Protection:** Intercepts prompts in DOM memory before network packets are formatted.
- **Zero Data Exfiltration:** 100% of detection runs on client CPU. Zero remote servers.

14. Setup & 15. Testing Benchmarks

Local Execution Commands

```
# 1. Run 95-prompt automated evaluation benchmark
node eval-harness.js

# 2. Run 121 unit tests
node tests/test-detection-engine.js
```

Evaluation Benchmark Results (95 Test Prompts)

Metric	Score	Benchmark Standard
Total Prompts Evaluated	95 Prompts	34 Clean, 18 API Keys, 11 Passwords, 19 PII, 13 Confidential
Accuracy	**100% (1.0000)**	61 True Positives, 34 True Negatives
Precision	**100% (1.0000)**	0 False Positives
Recall	**100% (1.0000)**	0 False Negatives
F1 Score	**1.0000**	Perfect harmonic balance
False Positive Rate	**0.00%**	Clean code & educational queries pass freely

16. Results, 17. Limitations & 18. Future Scope

- **Completed:** Manifest V3 extension, 150+ signatures, Shannon entropy, Luhn verification, HUD warning banner, one-click auto-redact, bulk data detector, explainability drawer, tamper-proof SHA-256 audit log, 25+ AI platform support.
- **Limitations:** Scoped to Chromium browsers; monitors browser-based interactions (terminal curl requests are outside browser scope).
- **Future Scope:** WebAssembly OCR for multimodal image uploads, enterprise SIEM webhook streaming, and Firefox/Safari WebExtension ports.

20. Viva Examination Preparation (30 Q&A)

1. What is "Shadow AI" and why is it dangerous?

Shadow AI refers to employees using unauthorized public AI chatbots (like ChatGPT) for work tasks. It is dangerous because pasting proprietary code, API keys, or customer data into external AI tools risks data leaks and regulatory penalties.

2. What is the primary purpose of SENTINEL?

SENTINEL is a browser extension that inspects prompts on AI websites in real time, blocking sensitive credentials, PII, and trade secrets before they leave the browser.

3. What does "Client-Side Zero-Trust" mean in your project?

100% of detection logic, regex matching, entropy calculation, and logging happens locally in the browser. Zero prompt data or telemetry is ever transmitted to any external server.

4. Why is blocking data at the browser level better than using a network proxy?

A network proxy requires breaking TLS encryption, introduces network latency, and requires managing complex proxy servers. SENTINEL intercepts text in the DOM before network packets are even created.

5. Which AI platforms does SENTINEL support?

Over 25 major platforms, including ChatGPT, Anthropic Claude, Google Gemini, Microsoft Copilot, Perplexity, Groq, Mistral, HuggingFace Chat, Poe, and Cursor.

6. Why did you build SENTINEL using Chrome Manifest V3 instead of V2?

Manifest V3 is Google's modern standard for browser extensions. It replaces persistent background pages with lightweight background service workers and enforces stricter security models.

7. What is the difference between an isolated world content script and the main world?

Content scripts run in an isolated execution world—they share the DOM with the webpage but cannot access the page's global JS variables. We inject `main-world-intercept.js` into the main world to intercept the page's actual `fetch` and `XHR` calls.

8. How do the content script and main-world script communicate?

They communicate asynchronously across the window boundary using `window.postMessage()` with custom event types (`SAPG_CHECK_PROMPT` and `SAPG_BLOCK_DECISION`).

9. What database is used in this project?

`chrome.storage.local` , an asynchronous, quota-managed NoSQL local key-value store built into the browser.

10. Why did you use Vanilla JavaScript and CSS instead of React or Tailwind?

Extensions must be extremely fast and lightweight. Vanilla JS and CSS eliminate framework bundle size, require zero build tools, load in less than 1 millisecond, and prevent CSS conflicts with the host webpage.

11. How does the Shannon Entropy algorithm work in your project?

Shannon entropy measures character randomness in a string: $H = -\sum p_i \log_2 p_i$. High entropy (≥ 4.0) on tokens 20+ characters long flags them as likely API keys or secrets, even without known prefixes.

12. How does your Auto-Redact feature avoid corrupting text indexes?

It sorts all detected findings in descending order by starting character position (`b.start - a.start`) and replaces them from the end of the text to the beginning.

13. Why do you need special logic to update text in ChatGPT?

React 18 overrides standard HTML property setters. We invoke the native prototype descriptor setter (`HTMLTextAreaElement.prototype`) and dispatch synthetic `input` and `beforeinput` events to sync React's internal state.

14. What is the Luhn Algorithm and where is it used?

The Luhn algorithm (MOD-10) verifies credit card checksums. We use it in `luhnCheck()` to ensure random 16-digit numbers in code aren't falsely flagged as credit cards unless mathematically valid.

15. How does your system detect bulk data exports?

`detectBulkData()` counts instances per PII type. If 5 or more of the same type (e.g. 6 emails) or 10+ mixed PII items are found, it generates a "Bulk Data Export" critical finding.

16. How does your audit log achieve "tamper-proof" integrity?

It uses cryptographic SHA-256 hash chaining: each audit record contains the hash of the previous record. Modifying any past record breaks the chain head hash stored in `sapg_audit_chain` .

17. How does the system handle obfuscation or bypass attempts (e.g. "s k - a b c")?

`normalizeObfuscated()` strips zero-width invisible characters, normalizes Unicode homoglyphs, converts leet-speak numbers, and collapses single-character spacing before running detection.

18. How does base64 detection work?

It matches base64 patterns ≥ 32 characters, decodes them using `atob()` , and re-scans the decoded plaintext through the API key detection engine.

19. What compliance presets are implemented?

HIPAA (Medical Record Numbers, NPI, PHI keywords), PCI-DSS (Payment cards, track data, CVV), GDPR (Data subject rights, citizen IDs), and SOC 2 (audit and access control terms).

20. How does the engine prevent false positives on educational queries?

`isEducationalContext()` checks for question prefixes like *"What is..."* or *"Explain..."* and applies a 50% risk discount to API keys/passwords while keeping PII strictly protected.

21. Why didn't you use chrome.webRequest blocking in your service worker?

In Manifest V3, synchronous blocking via `webRequestBlocking` is deprecated and unavailable. Content script + main-world interception is the official and most reliable MV3 approach.

22. Why did you choose Regex + Shannon Entropy over a heavy Machine Learning model?

Regex and Shannon entropy execute deterministically in 0 to 20 milliseconds with 100% precision and zero memory overhead. A full deep learning model would consume 100MB+ of RAM and cause typing lag.

23. Why do you store masked previews instead of encrypted secrets in the audit log?

Under zero-trust principles, the extension should never persist raw credentials on disk. Storing masked strings (e.g. `AKIA••••••••••••LE`) provides audit context without creating a credential cache for malware to steal.

24. Why did you implement both inline highlight overlays and HUD banners?

The inline overlay provides gentle, non-blocking real-time awareness while typing, whereas the HUD banner acts as a hard stop during submission to prevent accidental exfiltration.

25. What happens if an AI chat app sends data via WebSockets?

SENTINEL's DOM interception layer (Form submit, Enter keypress, Send button click) runs in the capture phase before the application's internal handlers can dispatch data across any network protocol.

26. How do you handle Single Page Applications (SPAs) where forms dynamically re-render?

We attach a `MutationObserver` to `document.body` that listens for DOM tree mutations and automatically attaches listeners to newly created textareas and contenteditable elements.

27. What is the time complexity of your detection engine?

$O(N)$ linear time with respect to prompt character length N, because regular expressions and Shannon entropy process the input text in single passes over token streams.

28. Can a malicious script inside the webpage tamper with SENTINEL's postMessage bridge?

No, because `content.js` runs in an isolated V8 execution context. It validates message origins and schema structures, while critical blocking logic occurs within the isolated content script.

29. What are the benchmark results of your evaluation harness?

Across 95 test prompts (34 clean, 61 sensitive), SENTINEL achieved 100% accuracy, 100% precision, 100% recall, 1.00 F1 score, and 0% false positive rate.

30. If you had 6 more months, what major feature would you add?

Local OCR via WebAssembly to inspect images and screenshots pasted into multimodal models (like GPT-4o and Claude 3.5 Sonnet) for embedded credentials.

21. 2-Minute Faculty Pitch & 22. 5-Minute Presentation Script

2-Minute Spoken Explanation for Faculty:

*"Good morning respected faculty. Today I present **Shadow AI Prompt Guardian — SENTINEL**, a client-side Data Loss Prevention browser extension built on Chrome Manifest V3. As AI chatbots like ChatGPT and Claude become daily tools, employees frequently paste sensitive source code, customer records, database credentials, and production API keys directly into prompts. This 'Shadow AI' usage leads to critical data leaks and regulatory penalties under GDPR and HIPAA. Existing DLP solutions use heavy network proxies that slow down connections and compromise user privacy. In contrast, SENTINEL operates **100% locally in the browser client** with zero external network requests. SENTINEL intercepts text before it leaves the browser using a 5-vector interception pipeline hooking DOM events and network APIs. Our multi-layer detection engine combines over 150 signature regexes, Shannon entropy calculation for unknown secret tokens, Luhn validation for payment cards, and custom keyword dictionaries. If a violation is detected, SENTINEL halts the request, displays a holographic warning HUD with an explainability drawer, and offers a **One-Click Auto-Redact** feature that replaces sensitive secrets with safe placeholders like `[API_KEY]` while updating the chatbot's React state seamlessly. On our automated benchmark of 95 synthetic scenarios, SENTINEL achieved **100% accuracy, 100% precision, and zero false positives** with an average execution latency of under 25 milliseconds. All incident logs are protected on-disk using SHA-256 cryptographic hash chains. Thank you, and I am ready for your questions."*

★ WHAT I MUST KNOW FOR VIVA (Top 20 Summary Sheet)

1. **The Core Problem:** Employees accidentally leak API keys, source code, and customer PII into public AI chatbots ("Shadow AI"), risking major security breaches and regulatory fines.
2. **The Core Solution:** SENTINEL is a 100% client-side Chrome Extension (Manifest V3) that inspects prompts locally and blocks sensitive submissions before network requests leave the browser.
3. **Zero Data Exfiltration:** No prompt data, tokens, or telemetry ever leave the user's browser. Everything runs locally in JavaScript.
4. **Why Manifest V3:** MV3 is Chrome's modern standard. It uses lightweight background service workers and strictly scopes permissions (`storage`, `activeTab`).
5. **Main-World vs Isolated World:** Content scripts cannot access the page's JS variables. We inject `main-world-intercept.js` into the page's main world to monkeypatch native `fetch` and `XMLHttpRequest`, communicating back to the content script via `window.postMessage`.
6. **The 5 Interception Vectors:** Form `submit` event, Enter key `keydown`, Send button `click`, `window.fetch`, and `XMLHttpRequest.prototype.send`.
7. **Shannon Entropy Formula:** $H(X) = -\sum P(x) \log_2 P(x)$. Measures character randomness to catch unknown secrets ≥ 20 chars with $H \ge 4.0$.
8. **Deterministic Pattern Engine:** Over 150 high-precision regular expressions covering AWS, GitHub, OpenAI, Anthropic, Stripe, GCP, JWTs, private keys, database connection strings, and PII.
9. **Luhn Algorithm Validation:** Prevents false positive credit card alerts by running candidate numbers through the MOD-10 check digit calculation.
10. **One-Click Auto-Redaction:** Slices sensitive spans from the tail end (descending index) to preserve character offsets, replacing them with `[API_KEY]`, `[PASSWORD]`, or `[REDACTED]`.
11. **React State Synchronization:** Directly setting `textarea.value` fails in React 18. We invoke the native prototype setter descriptor and dispatch synthetic `input` / `beforeinput` events.
12. **Severity Levels:** Critical (API keys, passwords, bulk PII — always blocks), Warning (single PII), and Info (confidential terms).
13. **Bulk Data Detection:** Prompts containing 5+ instances of the same PII type (e.g. 6 emails) or 10+ mixed items are flagged as Critical "Data Export" leaks.
14. **Explainability Drawer:** Clicking category chips opens a panel showing rule ID, severity, human reason, and character position—**never displaying raw secret text**.
15. **Real-Time Mirror Overlay:** Positions an invisible computed-style mirror div over textareas with colored `<mark>` elements on a 300ms debounce without disrupting typing.
16. **Live Risk Indicator:** Floating UI pill at the bottom corner that updates risk status (Safe, Low, High, Danger) on a 400ms debounce.
17. **Tamper-Proof Audit Logging:** Local logs in `chrome.storage.local` are linked via SHA-256 cryptographic hash chains ($H_k = \text{SHA-256}(H_{k-1} \parallel \text{entry})$).
18. **Compliance Presets:** Domain-specific pattern sets for HIPAA (healthcare PHI), PCI-DSS (cards), GDPR (EU data), and SOC 2.
19. **Obfuscation Normalizer:** Strips zero-width invisible characters, converts leet-speak, maps Unicode lookalikes, and collapses character spacing ("s k - a b c").
20. **Benchmark Results:** Tested against 95 synthetic scenarios with **100% Accuracy, 100% Precision, 100% Recall, 1.00 F1 Score, and 0% False Positive Rate** at an average latency under 25ms.